

Este documento cataloga las medidas de seguridad activas en el API Gateway, analiza las practicas de proteccion implementadas en cada capa y documenta la superficie de ataque conocida.

1. Autenticacion JWT (Capa de Identidad)

El middleware `JwtAuthPlugin` implementa verificacion de tokens JSON Web Token en las rutas que lo requieran.

Protecciones implementadas:

- **Verificacion criptografica de firma:** Utiliza la libreria `jose` (sin dependencias nativas) para validar firmas HMAC con los algoritmos `HS256`, `HS384` y `HS512`.
- **Validacion de expiracion:** `jose.jwtVerify()` rechaza automaticamente tokens expirados basandose en el claim `exp`.
- **Sanitizacion de headers anti-spoofing:** Antes de procesar el token, el middleware elimina cualquier header entrante que comience con el prefijo `x-jwt-claim-`. Esto previene que un atacante inyecte claims falsificados directamente desde el cliente, suplantando la identidad de un usuario autenticado.
- **Inyeccion segura de claims al backend:** Solo se inyectan claims escalares (`string`, `number`, `boolean`) como headers normalizados en lowercase. Valores complejos (objetos, arrays) se descartan silenciosamente para evitar ataques de inyeccion de headers.

Superficie de atencion:

- Los secretos JWT se almacenan en el archivo `gateway.yaml` como texto plano. En produccion, se recomienda inyectarlos mediante variables de entorno con la sintaxis `\${JWT_SECRET}` y gestionarlos a traves de un gestor de secretos (AWS Secrets Manager, HashiCorp Vault, etc.).
 - Solo se soportan algoritmos HMAC simetricos. No se implementa soporte para RS256/ES256 (claves asimetricas).
-

2. Rate Limiting (Capa de Proteccion contra Abuso)

El middleware `RateLimitPlugin` implementa limitacion de tasa basada en ventana de tiempo fija, respaldado por Redis para entornos distribuidos.

Protecciones implementadas:

- **Limitacion por IP de origen:** Las cuotas se evaluan por direccion IP del cliente, identificada a traves de los headers `x-forwarded-for`, `x-real-ip` o la IP del socket remoto.
- **Ventana de tiempo configurable:** Cada ruta puede definir `maxRequests` y `windowSeconds` de forma independiente.
- **Degradacion segura (Fail-Open):** Si Redis no esta disponible y `redis.onFailure` esta configurado como `"open"`, las peticiones no se bloquean. El trafico continua fluyendo para evitar una interrupcion total del servicio.

Superficie de atencion:

- El modo `fail-open` permite trafico ilimitado cuando Redis cae. En entornos de alta seguridad, considerar configurar `redis.onFailure: \"closed\"` para rechazar todas las peticiones cuando el store no este disponible.
 - No se implementa Rate Limiting por clave de API, usuario autenticado o ruta especifica mas alla del prefijo.
-

3. Circuit Breaker (Capa de Resiliencia de Red)

El middleware `CircuitBreakerPlugin` implementa el patron de interruptor de circuito para proteger tanto al Gateway como a los backends de fallos en cascada.

Protecciones implementadas:

- **Aislamiento de fallos por ruta:** Cada ruta tiene su propia maquina de estados independiente (CLOSED, OPEN, HALF_OPEN).
 - **Deteccion de fallos por umbral porcentual:** El circuito se abre cuando el porcentaje de errores en la ventana de evaluacion supera el `errorThreshold` configurado.
 - **Deteccion por fallos consecutivos:** El circuito se abre inmediatamente despues de 5 fallos consecutivos, sin esperar a alcanzar el tamaño de la ventana.
 - **Recuperacion gradual (Half-Open):** Tras el periodo de `recoveryTimeMs`, se permite un numero controlado de peticiones de prueba (`halfOpenRequests`). Si todas son exitosas, el circuito se cierra. Si alguna falla, se reabre.
 - **Reintentos con backoff exponencial:** Las peticiones fallidas se reintentan automaticamente con delay exponencial limitado por `retryMaxDelayMs`.
-

4. Proxy y Comunicaciones de Red

Protecciones implementadas:

- **Headers de forwarding controlados:** El motor de proxy (`ProxyEngine`) construye los headers de forwarding (`x-forwarded-for` , `x-forwarded-proto` , `x-forwarded-host`) de forma explicita, sin permitir la propagacion de headers arbitrarios del cliente que podrian confundir al backend.
 - **Request ID unico:** Cada peticion recibe un `x-request-id` generado internamente para trazabilidad end-to-end.
 - **Timeouts estrictos:** Cada ruta puede definir timeouts independientes para `connect` , `headers` y `body` , evitando que una peticion lenta consuma recursos indefinidamente.
 - **Connection Pooling con limites:** Undici gestiona pools de sockets con limites de conexiones concurrentes, previniendo el agotamiento de descriptores de archivo del sistema operativo.
-

5. Configuracion y Datos Sensibles

Protecciones implementadas:

- **Inmutabilidad de la configuracion:** La configuracion cargada se congela recursivamente con `Object.freeze()` . Ningun modulo puede mutar accidentalmente los valores de configuracion en tiempo de ejecucion.
- **Validacion estricta con Zod:** Todos los campos de configuracion se validan en el arranque contra esquemas tipados. Un campo malformado o faltante detiene el proceso inmediatamente antes de aceptar trafico.

- **Interpolacion segura de variables de entorno:** La sintaxis `\${VAR}` solo busca variables del proceso actual. Si una variable referenciada no existe, el arranque falla con `MissingEnvVarError`, evitando valores vacios silenciosos.

Superficie de atencion:

- El archivo `.env.example` no contiene secretos reales, pero el archivo `.env` de produccion debe protegerse mediante permisos de sistema de archivos restrictivos (600/400).
- El `gateway.yaml` de ejemplo contiene un secreto JWT de prueba en texto plano. Este valor no debe usarse en produccion.

El Gateway define una jerarquia de errores personalizados para evitar la exposicion de informacion interna en las respuestas HTTP:

Error	Codigo HTTP	Contexto
<code>RouteNotFoundError</code>	404	No se encontro una ruta que coincida con el path solicitado.
<code>RateLimitError</code>	429	Se excedio el limite de peticiones para esta IP en la ventana de tiempo.
<code>BackendError</code>	502	Error de comunicacion con el servicio de destino (conexion rechazada, timeout, reset).
<code>ConfigError</code>	500	Error de configuracion (solo ocurre durante el arranque, nunca en produccion estable).

Todas las respuestas de error incluyen un campo `timestamp` ISO 8601 para correlacion con logs pero no exponen stack traces ni detalles internos del sistema.

Medida	Estado	Detalle
Autenticacion JWT	Activa (por ruta)	Verificacion HMAC con sanitizacion de headers.
Rate Limiting	Activo (por ruta)	Ventana fija respaldada por Redis.
Circuit Breaker	Activo (por ruta)	Maquina de estados con reintentos exponenciales.
Sanitizacion de Headers	Activa	Eliminacion de headers <code>x-jwt-claim-*</code> del cliente.
Inmutabilidad de Config	Activa	Deep freeze de objetos de configuracion.
Validacion de Esquemas	Activa	Zod con mensajes descriptivos en espanol.
CORS	No implementado	No existe middleware de CORS en el Gateway.

Medida	Estado	Detalle
CSP	No aplica	El Gateway es un servicio backend; no sirve HTML.
HTTPS/TLS	Delegado	Se espera que un balanceador de carga o ingress controller maneje TLS upstream.